

빈도와 그룹 집계

value_counts · pd.cut · groupby

전체 평균 하나로써는 어느 라인이 문제인지 알 수 없습니다 — 그룹으로 나눠야 보입니다

13_02 마무리 (실습 5~7 · copy) · 14_01 빈도와 그룹 집계

핵심 키워드

범주형 vs 수치형 · `.value_counts()` · `normalize=True` · 불균형 데이터

`pd.cut(bins, labels)` · 구간 빈도 · `.sort_values()` · `.copy()`

분할-적용-결합 · `.groupby("열")` · 여러 키 그룹 · `.size()` · `.count()` · `numeric_only=True`

2026년 8월 18일 (화) · 20일차 · 50차시

국립금오공과대학교 컴퓨터공학부 박민호

이 책을 읽는 법

뱃지 · 자료 고지 · 지난 회차와의 연결

1. 항목 뱃지

오늘은 **메서드**가 대부분입니다. `.value_counts()`, `.groupby()`, `.size()` 모두 점을 찍고 부릅니다. 예외는 `pd.cut()` 하나로, 이것만 `pd`가 앞에 오는 함수입니다.

뱃지	판별법	이번 회차 예
메서드	점 + 이름 + 소괄호	<code>.value_counts()</code> · <code>.groupby()</code> · <code>.size()</code> · <code>.copy()</code>
함수	앞에 점이 없음 — 모듈 이름이 앞에	<code>pd.cut()</code>
키워드	괄호 안의 이름=값	<code>normalize=</code> · <code>bins=</code> · <code>labels=</code> · <code>numeric_only=</code>
자료형	값의 종류	범주형 · 수치형 · <code>DataFrameGroupBy</code>
개념	문법이 아닌 사고방식	분할-적용-결합 · 불균형 데이터

2. 이번 회차 자료 고지

항목	상태	이 책에서
교안 14_01	있음 (40쪽)	STEP 2~7의 뼈대
강사 코드	13_02_14~17 · 14_01_01~04	오늘 배포분 그대로 반영
groupby 코드	배포본에 없음	교안 p22~p35 + 전사본으로 복원
전사본(STT)	있음 · 88,751자	타임스탬프가 없어 시간 구간은 확인 불가
교안 예상 결과	실데이터와 불일치	부록 B-3에 대조표

주의

groupby 시연 코드가 배포본에 없습니다. 강사님이 전사본 후반에 "이 코드는 쉬는 시간 끝나고 본격적으로 만들어서 보여 드리겠다"고 하셨고, 오늘 배포된 14_combined.py는 14_01_01~14_01_04(value_counts · cut · 실습 1~3)까지입니다. groupby 부분은 교안 p22~p35와 전사본, 그리고 직접 푸신 14_01_example_advanced.py로 복원했습니다.

3. 지난 회차와의 연결

오늘은 19일차(8/14)의 남은 실습을 마무리하고 새 단원으로 넘어간 날입니다. 8월 17일이 광복절 대체휴일이라 나흘 만의 수업이었고, 강사님도 "지난주 금요일에 일찍 자리를 뜨신 분들이 계시니 필터링 이야기를 조금 더"라며 복습으로 시작했습니다.

19일차 (8/14)	20일차 (오늘)	관계
sort_values 개념 설명	실습 5·6·7 코드 시연	배포 코드가 오늘 나왔습니다
.copy() 개념	df_bad['품질등급'] = '점검' 실제 수정	경고 없이 값이 바뀌는 것을 확인
df[df['등급']=='불량'] 한 조건	.value_counts() 모든 값을 한 번에	조건을 일일이 거는 대신
조건.mean() 참 비율	normalize=True 값별 비율	같은 발상의 확장
불량만 골라 평균	groupby 등급별 평균을 한 번에	19일차 예고가 오늘 실현
18일차 대괄호 한 겹·두 겹	groupby(...)["온도"] vs [["온도", "진동"]]	같은 규칙이 그대로

CONTENTS

목차

오늘 배운 것 전체 지도

장 · STEP	무엇을 배우나	핵심
CHAPTER 0	오늘의 지도 — 집계로 답하는 세 가지 질문	빈도 · 그룹 비교 · 관계
STEP 1	지난 시간 마무리 — 실습 5·6·7과 copy	sort_values · .copy()
STEP 2	범주형과 수치형 — 집계의 갈림길	바로 세기 vs 구간 필요
STEP 3	value_counts — 값별 개수 세기	.value_counts()
STEP 4	normalize — 개수를 비율로	normalize=True · 불균형
STEP 5	pd.cut — 수치형을 구간으로 묶기	bins · labels
STEP 6	groupby — 분할·적용·결합	전체 평균의 함정
STEP 7	groupby 실전 — 여러 키와 size	groupby(["A", "B"]) · .size()
부록 A	치트시트 — 빈도·구간·그룹 도구 요약	한 장 요약
부록 B	오류·함정 + 제출 코드 교정 + 교안 예상 결과 대조	X → ✓
부록 C	실습 하이라이트 · 학생 데이터 종합 · 다음 시간	실습 4~7 예고

오늘 하루의 흐름 — 거르기에서 요약하기로

지금까지는 원하는 행을 골라내는 일을 배웠습니다. 오늘부터는 골라낸 것을 숫자 몇 개로 요약하는 일을 배웁니다. 교안 p5의 표현대로 "집계란 여러 값을 한 숫자로 압축하는 것" 이고, 모든 데이터 분석의 출발점입니다.



오늘의 지도

집계로 답하는 세 가지 질문

설비 적용

오늘부터 새 데이터입니다. **유압 시스템 상태 감시**(14_hydraulic.csv, 120행 8열)로, 유압 시스템이 60초 부하 사이클을 반복하며 압력·온도·진동을 측정한 기록입니다. **범주형** 열이 냉각기상태(정상/저하/고장)·운전부하(고/저)·밸브상태(정상/경미/지연/심각)이고, **수치형** 열이 온도·진동·압력·냉각효율입니다. 냉각기가 정상이면 온도 약 36도, 저하면 45도, 고장이면 55도로 **상태별로 뚜렷이 갈리는** 데이터라 그룹 집계 연습에 알맞습니다.

0-1. 집계로 답할 수 있는 세 가지 질문

교안 p6이 이번 단원 전체를 세 질문으로 묶습니다. 오늘은 앞의 두 가지를 배우고, 세 번째는 다음 시간입니다.

질문	무엇을 묻나	도구	언제
① 빈도	어떤 값이 가장 많은가	<code>value_counts · pd.cut</code>	오늘
② 그룹 비교	그룹마다 평균이 다른가	<code>groupby</code>	오늘
③ 관계	두 값이 함께 움직이는가	<code>corr</code>	다음 시간 (14_03)

개념

교안 p5가 이유를 명확히 합니다 — "**행을 하나씩 읽는 대신 요약해 전체를 파악합니다.**" 공정 데이터는 행이 수천 개라 눈으로 읽는 것이 불가능합니다. 그래서 데이터를 받으면 **가장 먼저 전체를 압축**하는 것이 분석의 첫 동작입니다.

0-2. 오늘 나온 이름 전체

분류	이름	한 줄
개념	범주형 (categorical)	종류를 나타내는 값 — 빈도 집계 대상
개념	수치형 (numerical)	측정한 숫자 — 평균·합계 가능, 빈도는 구간이 필요
메서드	<code>.value_counts()</code>	값별 개수 세기 — 많은 순으로 자동 정렬
키워드	<code>normalize=True</code>	개수 대신 비율로

분류	이름	한 줄
키워드	<code>sort=False</code>	등장 순서로 표시 (기본은 많은 순)
키워드	<code>dropna=False</code>	결측도 하나의 종류로 셈
함수	<code>pd.cut(열, bins, labels)</code>	수치형을 구간으로 묶기
개념	불균형 데이터	한쪽이 압도적으로 많은 데이터 — 정확도의 함정
메서드	<code>.groupby("열")</code>	그룹으로 나누기 — 결과의 줄 수를 정함
개념	분할-적용-결합	groupby가 작동하는 3단계
메서드	<code>.size()</code>	그룹별 전체 행 수 — 결측 포함
메서드	<code>.count()</code>	그룹별 비결측 값 수 — 열 지정 필요
키워드	<code>numeric_only=True</code>	숫자 열만 집계 — 글자 열 오류 방지

STEP 1

지난 시간 마무리

실습 5·6·7과 copy

오늘 앞부분은 **19일차에 개념만 배우고 코드로 확인하지 못한 부분**을 마저 짚는 시간이었습니다. 강사님이 배포한 코드가 13_02_14부터 13_02_17까지 네 개인데, 각각 실습 5·6, copy, 실습 7에 해당합니다.

1-1. 실습 5 — 정렬과 다중 정렬

강사 코드 13_02_14

```
import pandas as pd

df = pd.read_csv("data/13_diecasting_shot.csv", encoding="utf-8")
print(df.shape)

# 비스킷두께 내림차순 상위 5개
print(df.sort_values("비스킷두께", ascending=False).head(5))

# 여러 열을 리스트로 묶어 우선순위 다중 정렬
df_multi = df.sort_values(["품질등급", "형체력"], ascending=[True, False])
print(df_multi.head(5))
```

▶ (200, 7)

연결

19일차 STEP 7에서 배운 **다중 키 정렬**입니다. ["품질등급", "형체력"]은 품질등급으로 큰 덩어리를 나누고 그 안에서만 형체력으로 세부 정렬합니다. ascending=[True, False]로 **열마다 방향을 다르게** 준 것도 그때 배운 그대로입니다.

1-2. 실습 6 — 필터링과 정렬 연결

강사 코드 13_02_15 — 매서드 체이닝

```
# 거르고 → 정렬하고 → 상위 5개, 점으로 이어 붙이기
df_filtered = (df[df["품질등급"] == "불량"]
               .sort_values("비스킷두께", ascending=False)
               .head(5))
print(len(df[df["품질등급"] == "불량"]), "건 중 상위 5건")
```

▶ 20 건 중 상위 5건

1-3. copy — 원본을 건드리지 않고 수정하기

19일차 STEP 8에서 개념만 봤던 부분을 오늘 코드로 확인했습니다. 강사 코드의 주석이 명확합니다 — "원본에 변경을 주려면 꼭 .copy()를 하세요. 안 그러면 SettingWithCopyWarning 경고가 발생합니다."

강사 코드 13_02_16

```
import pandas as pd

df = pd.read_csv("data/13_diecasting_shot.csv", encoding="utf-8")

# .copy() 로 원본과 분리된 복사본을 만든 뒤
df_bad = df[df["품질등급"] == "불량"].copy()

# 값을 바꿔도 경고가 없습니다
df_bad["품질등급"] = "점검"
print(df_bad.head())

# 원본은 그대로
print(df["품질등급"].value_counts().to_dict())
```

▶ 샷 실린더압력 주조압력 사이클타임 비스킷두께 형체력 품질등급

▶ 180 181 108.0 522.0 652.3 14.0 222.0 점검

▶ 181 182 214.0 1036.0 93.1 12.0 247.0 점검

▶ 182 183 215.0 1041.0 21.3 4.0 258.0 점검

▶ {'양품': 162, '불량': 20, '주의': 18}

개념

마지막 줄이 중요합니다. 복사본의 품질등급을 전부 "점검"으로 바꿨는데 원본은 그대로입니다. `.copy()`가 원본과의 연결을 끊었기 때문입니다. 이것을 안 붙이면 경고가 뜨고, 최악의 경우 원본까지 바뀝니다.

1-4. 실습 7 — 워크플로우 5단계 종합

19일차 부록에 보충답안으로 넣었던 실습 7을 오늘 강사님이 직접 풀었습니다. 분석 워크플로우 5단계에 정확히 맞춰 짜여 있어 그대로 옮깁니다.

강사 코드 13_02_17 — 5단계 그대로

```
import pandas as pd

# 1. 불러오기
df = pd.read_csv("data/13_diecasting_shot.csv", encoding="utf-8")

# 2. 확인하기
df.info()

# 3. 필터링
df_warning = df[(df["비스킷두께"] >= 16) | (df["사이클타임"] >= 100)]
print(len(df_warning))

# 4. 정렬
df_report = df_warning.sort_values("비스킷두께", ascending=False)

# 5. 선택 — 대괄호 중첩 주의
df_final = df_report[["샷", "품질등급", "형체력", "사이클타임"]]
print(df_final.head(3))
```

```
▶ 76
▶   샷 품질등급  형체력 사이클타임
▶ 197   198   불량 354.0    36.6
▶ 191   192   불량 366.0    36.9
▶  42    43   주의 253.0    20.7
```

주의

강사 코드 5단계 주석에 "대괄호 중첩 주의" 라고 적혀 있습니다. 18일차에 배운 두 겹 대괄호입니다. 열 여러 개를 고를 때 `df[["A", "B"]]`처럼 반드시 두 겹으로 써야 하고, 한 겹으로 쓰면 튜플로 해석돼 `KeyError`가 납니다.

STEP 2

범주형과 수치형

집계의 갈림길

여기서부터 교안 14_01입니다. 집계를 배우기 전에 **열의 종류를 구분하는 것**이 먼저입니다. 종류에 따라 쓸 수 있는 집계기가 달라지기 때문입니다.

구분	수치형 (numerical)	범주형 (categorical)
무엇인가	숫자로 측정한 값	종류를 나타내는 값
이 데이터의 예	온도 · 진동 · 압력 · 냉각효율	냉각기상태 · 운전부하 · 밸브상태 · result
가능한 집계	평균 · 합계 · 표준편차	빈도(개수 세기)
빈도 집계	구간으로 묶어야 의미가 생김	바로 가능
도구	pd.cut 후 value_counts	value_counts

2-1. 왜 수치형은 바로 못 세는가

온도 열에 value_counts를 바로 붙이면 어떻게 될까요? 실제로 해 보면 이유가 바로 보입니다.

같은 데이터, 다른 결과

```
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

```
# 범주형 — 바로 셀 수 있습니다
```

```
print(df["냉각기상태"].value_counts().to_dict())
```

```
# 수치형 — 고유값이 너무 많습니다
```

```
print("온도 고유값 개수:", df["온도"].nunique())
```

```
▶ {'고장': 40, '저하': 40, '정상': 40}
```

```
▶ 온도 고유값 개수: 62
```

120행짜리 데이터인데 **온도의 고유값이 62가지**입니다. 하나하나 세면 62줄짜리 표가 나오고, 대부분 개수가 1~2건이라 아무 정보가 없습니다. 강사 코드 주석 그대로 **"범위 없이 개별 경우의 수를 따지면 62가지나 되어 버린다"**는 것입니다. 그래서 수치형은 **먼저 묶고, 그 다음에 셉니다** — 35~40도는 낮음, 40~50도는 보통처럼 구간으로 나누면 62줄이 3줄로 줄어듭니다. 이 작업을 **범주화**라고 부르고, 도구가 pd.cut입니다(STEP 5).

STEP 3

value_counts

값별 개수 세기

주의

숫자가 들어 있어도 이름이면 범주형입니다. 설비 번호 M01·M04, 라인 번호 1·2·3처럼 계산할 의미가 없는 숫자가 그렇습니다. 판단 기준은 자료형이 아니라 **쓰임**입니다 — 평균을 내서 의미가 있으면 수치형, 없으면 범주형입니다. 학생 데이터의 학년이 정수형이지만 범주형인 것이 그 예입니다.

3-1. 조건을 일일이 거는 대신

강사님이 value_counts를 소개한 방식이 좋았습니다. **먼저 19일차 방식으로 해 보고**, 그것이 비효율적임을 확인한 뒤에 새 도구를 꺼냈습니다.

강사 코드 14_01_01

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

19일차 방식 — 조건을 하나씩

```
df_old = df[df["냉각기상태"] == "고장"]
print(len(df_old))
```

하지만 이 방식으로 모든 상태를 일일이 찾는 것은 비효율적입니다

"고장" 외의 모든 경우를 한 번에 세려면?

```
print(df["냉각기상태"].value_counts())
```

```
▶ 40
▶ 냉각기상태
▶ 고장 40
▶ 저하 40
▶ 정상 40
▶ Name: count, dtype: int64
```

메서드 `.value_counts()` 한 열의 값별 개수를 자동으로 세기

정의 이름 그대로 **value(값)**를 **counts(센다)** 는 뜻입니다. 세고 싶은 열을 고른 뒤 점을 찍어 붙입니다.

형태 `df["열이름"].value_counts()`

```
# 냉각기상태별 사이클 건수
print(df["냉각기상태"].value_counts().to_dict())
```

```
# result 열의 정상·고장 건수
print(df["result"].value_counts().to_dict())
```

```
# 밸브상태 — 종류가 네 가지
print(df["밸브상태"].value_counts().to_dict())
```

```
▶ {'고장': 40, '저하': 40, '정상': 40}
▶ {'정상': 67, '고장': 53}
▶ {'정상': 61, '지연': 20, '경미': 20, '심각': 19}
```

왜 쓰나 단 한 줄로 120개 행을 두세 개의 숫자로 요약합니다. 조건을 종류마다 거는 것과 비교하면 코드 길이가 몇 분의 일로 줄어듭니다.

응용 출력은 많은 순서로 자동 정렬됩니다. 밸브상태 결과를 보면 정상 61이 맨 위, 심각 19가 맨 아래입니다. 어떤 값이 가장 흔한지 따로 정렬할 필요 없이 바로 보입니다.

```
# 모든 범주형 열을 한 번에 훑기
for col in ["냉각기상태", "운전부하", "밸브상태", "result"]:
    vc = df[col].value_counts()
    print(f"{col:8s} 종류 {len(vc)}가지 · 최다 {vc.index[0]} {vc.iloc[0]}건")
```

```
▶ 냉각기상태  종류 3가지 · 최다 고장 40건
▶ 운전부하   종류 2가지 · 최다 고부하 60건
▶ 밸브상태  종류 4가지 · 최다 정상 61건
▶ result     종류 2가지 · 최다 정상 67건
```

자주 하는 실수

df 전체가 아니라 한 열만 골라낸 뒤 **점을 찍어야 합니다**. df.value_counts()도 동작하기는 하지만 **행 전체가 같은 조합인 경우를 세는** 전혀 다른 기능이라 원하는 결과가 나오지 않습니다.

3-2. 기본 동작과 옵션 두 가지

교안 p14가 강조합니다 — "**옵션을 외우기 전에 기본 동작이 무엇인지 먼저**" 입니다. 기본을 바꾸고 싶을 때만 옵션을 씁니다.

기본 동작	바꾸는 옵션	언제 쓰나
많은 순서로 정렬	<code>sort=False</code>	등급 A·B·C·D처럼 정해진 순서로 비교할 때
빈 값(NaN)은 무시	<code>dropna=False</code>	빈 칸의 개수도 중요한 정보일 때

설비 적용

교안 p13이 짝습니다 — "설비 데이터의 빈 칸은 센서 고장·통신 오류의 신호이고, 어느 그룹에 몰렸는지가 단서"입니다. 19일차에 13_diecasting_shot.csv에서 결측 14건을 찾았던 것이 그 예입니다. `dropna=False`로 세면 결측도 하나의 종류로 표에 나타납니다. 참고로 오늘 쓰는 유압 데이터는 결측이 하나도 없습니다.

옵션 확인

옵션은 쉼표로 이어 여러 개를 함께 쓸 수 있습니다

```
print(df["밸브상태"].value_counts(sort=False).to_dict())
print(df["밸브상태"].value_counts(dropna=False).to_dict())
```

결측이 있는 데이터라면 차이가 납니다

```
other = pd.read_csv("data/13_diecasting_shot.csv")
print("dropna 기본 :", other["비스킷두께"].isna().sum(), "건 제외됨")
```

- ▶ {'정상': 61, '지연': 20, '경미': 20, '심각': 19}
- ▶ {'정상': 61, '지연': 20, '경미': 20, '심각': 19}
- ▶ dropna 기본 : 14 건 제외됨

STEP 4

normalize

개수를 비율로 바꾸기

개수만으로는 **많고 적음을 판단하기 어렵습니다**. 교안 p11이 든 예가 명확합니다 — 1공장 불량 50건, 2공장 30건이면 1공장이 나빠 보이지만, 생산량을 고려한 비율(1% vs 6%)을 보면 정반대입니다.

키워드 `normalize=True` 개수 대신 비율로 계산

정의 전체를 1로 놓고 각 값이 차지하는 몫으로 변환합니다. `normalize`는 **정규화**라는 뜻입니다.

형태 `df["열"].value_counts(normalize=True)`

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")

# 개수
print(df["result"].value_counts().to_dict())

# 비율
print(df["result"].value_counts(normalize=True).round(3).to_dict())
```

- ▶ {'정상': 67, '고장': 53}
- ▶ {'정상': 0.558, '고장': 0.442}

왜 쓰나 규모가 다른 대상을 비교할 때 필수입니다. 라인마다 생산량이 다른데 불량 건수만 비교하면 큰 라인이 항상 나빠 보입니다.

응용 소수점이 길게 나오므로 `.round()`를 이어 붙이는 것이 관례입니다. 강사 코드 주석대로 **"정규화 비율 결과는 그대로 쓰기보다 round 처리로 반올림할 때가 많습니다."** 백분율로 보고 싶으면 100을 곱합니다.

```
vc = df["result"].value_counts(normalize=True)

print("소수 3자리:", vc.round(3).to_dict())
print("백분율 :", (vc * 100).round(1).to_dict())

# 개수와 비율을 나란히
import pandas as pd
tbl = pd.DataFrame({"개수": df["result"].value_counts(),
                    "비율": vc.round(3)})
print(tbl)
```

```
▶ 소수 3자리: {'정상': 0.558, '고장': 0.442}
▶ 백분율 : {'정상': 55.8, '고장': 44.2}
▶   개수   비율
▶ result
▶ 정상   67   0.558
▶ 고장   53   0.442
```

개념

19일차에 배운 조건.mean()과 같은 원리입니다. 그때는 조건 하나의 참 비율이었고, 지금은 값 종류마다의 비율입니다.

4-1. 불균형 데이터 — 정확도의 함정

교안 p19가 이 단원에서 가장 중요한 실무 개념을 짚습니다. 품질 검사 데이터로 확인해 봅시다.

실습 2 — 불균형 확인

```
import pandas as pd

df_qc = pd.read_csv("data/14_hydraulic_qc.csv", encoding="utf-8")
print("전체:", len(df_qc), "건")

print(df_qc["검사결과"].value_counts().to_dict())
print(df_qc["검사결과"].value_counts(normalize=True).round(3).to_dict())
```

```
▶ 전체: 200 건
▶ {'합격': 188, '불합격': 12}
▶ {'합격': 0.94, '불합격': 0.06}
```

개념

불합격이 전체의 6%뿐입니다. 여기서 함정이 생깁니다 — 고장 예측 모델을 만들었는데 무조건 "합격"이라고만 찍어도 정확도가 94%로 나옵니다. 아무 일도 안 하는 모델이 훌륭해 보이는 것입니다. 교안의 표현대로 "빈도 확인이 이 함정을 피하는 첫 안전장치"입니다.

설비 적용

실제 제조 현장의 불량 데이터는 거의 항상 이런 모습입니다. 불량률이 1%도 안 되는 공정이 흔합니다. 그래서 모델 성능을 볼 때 정확도(accuracy)만 보면 안 되고, 정밀도(precision)·재현율(recall)을 함께 봐야 합니다. 나중에 예측 모델을 배울 때 다시 나오는 개념이고, 그 출발점이 오늘의 `value_counts(normalize=True)` 한 줄입니다.

주의

교안 실습 2는 "round로 비율을 소수점 셋째 자리까지 정리"라고 하는데, 제출 코드와 강사 코드 모두 `.round(1)`로 되어 있어 {합격 0.9, 불합격 0.1}이 나옵니다. 셋째 자리면 `.round(3)`이고 {0.94, 0.06}입니다. 6%라는 불균형을 보려면 셋째 자리가 필요합니다 — 첫째 자리로 자르면 10%로 반올림돼 실체가 가려집니다.

STEP 5

pd.cut

수치형을 구간으로 묶기

STEP 2에서 확인했듯 수치형은 바로 세면 의미가 없습니다. **먼저 묶고, 그 다음에 셉니다.** 순서가 핵심입니다.

STEP 1

pd.cut 으로 묶기

STEP 2

value_counts 로 세기

함수 `pd.cut(열, bins, labels)` 수치형을 정해진 경계에서 잘라 구간으로

정의 숫자를 경계에서 잘라 구간(범주)으로 바꿉니다. 결과는 구간 이름표가 담긴 Series입니다.

형태 `pd.cut(df["온도"], bins=[0, 40, 50, 200], labels=["낮음", "보통", "높음"])`

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")

band = pd.cut(df["온도"],
               bins=[0, 40, 50, 200],
               labels=["낮음", "보통", "높음"])

print(band.value_counts().to_dict())
print("고유값이", df["온도"].nunique(), "가지에서", band.nunique(), "가지로")

▶ {'낮음': 41, '보통': 40, '높음': 39}
▶ 고유값이 62 가지에서 3 가지로
```

왜 쓰나 62줄짜리 무의미한 표가 **3줄짜리 의미 있는 표**가 됩니다. 경계 기준은 설비의 정상 온도 범위 같은 **현장 지식**을 반영해 정합니다.

응용 구간을 나눈 뒤에도 `normalize=True`를 그대로 쓸 수 있습니다. 개수와 비율을 함께 보면 분포가 더 분명해집니다.

```
band = pd.cut(df["진동"], bins=[0.0, 0.6, 0.7, 10.0],
              labels=["약함", "보통", "강함"])

print("개수:", band.value_counts().to_dict())
print("비율:", band.value_counts(normalize=True).round(3).to_dict())
```

```
▶ 개수: {'보통': 55, '약함': 48, '강함': 17}
▶ 비율: {'보통': 0.458, '약함': 0.4, '강함': 0.142}
```

자주 하는 실수

경계(bins)는 이름표(labels)보다 반드시 하나 많아야 합니다. **경계 4개면 구간 3개입니다.** 강사 코드 주석에도 이 점이 「엣지」로 적혀 있습니다. 개수가 안 맞으면 ValueError가 납니다.

5-1. 경계를 정하기 전에 범위부터 확인합니다

교안 실습 3의 첫 단계가 "최솟값과 최댓값으로 값의 범위 확인"인 이유가 있습니다. 범위를 모르고 경계를 정하면 한 구간에 다 몰리거나 빈 구간이 생깁니다.

실습 3 — 범위 확인 후 구간 설정

```
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

① 범위 확인이 먼저

```
print("진동 범위:", df["진동"].min(), "~", df["진동"].max())
```

② 범위에 맞춰 경계 정하기

```
band = pd.cut(df["진동"], bins=[0.0, 0.6, 0.7, 10.0],
              labels=["약함", "보통", "강함"])
print(band.value_counts().to_dict())
```

```
▶ 진동 범위: 0.53 ~ 0.779
▶ {'보통': 55, '약함': 48, '강함': 17}
```

주의

진동의 실제 범위는 **0.53 ~ 0.779**인데 경계를 [0.0, 0.6, 0.7, 10.0]으로 잡았습니다. 양 끝(0.0과 10.0)이 실제 범위보다 훨씬 넓은데, 이것은 **의도적인 여유**입니다. 나중에 데이터가 늘어 0.9짜리 값이 들어와도 "강함"으로 잡히기 때문입니다. 경계를 실제 최솟값·최댓값에 딱 맞추면 새 데이터에서 구간 밖 값이 NaN이 됩니다.

경계는 왼쪽이 열려 있습니다

경계를 실제 범위에 딱 맞추면 위험합니다

```
tight = pd.cut(df["진동"], bins=[0.53, 0.6, 0.7, 0.779],
               labels=["약함", "보통", "강함"])
print("최솟값 0.53 이 구간에 들어갔나?", tight.isna().sum(), "건이 NaN")
```

여유를 두면 안전합니다

```
safe = pd.cut(df["진동"], bins=[0.0, 0.6, 0.7, 10.0],
              labels=["약함", "보통", "강함"])
print("여유를 둔 경우:", safe.isna().sum(), "건이 NaN")
```

▶ 최솟값 0.53 이 구간에 들어갔나? 1 건이 NaN

▶ 여유를 둔 경우: 0 건이 NaN

개념

pd.cut의 구간은 기본이 **왼쪽 열림·오른쪽 닫힘**((0.53, 0.6] 형태)입니다. 그래서 경계를 최솟값 0.53에 딱 맞추면 **0.53인 값 자체가 어느 구간에도 안 들어가 NaN 이 됩니다**. 최솟값보다 작은 값을 첫 경계로 두는 것이 **안전합니다**. 반대 동작이 필요하면 right=False 옵션을 씁니다.

STEP 6

groupby

분할 · 적용 · 결합

여기서부터 교안 14_01 PART 02입니다. 오늘의 마지막 주제이고, 앞으로 배울 모든 집계의 뼈대가 됩니다.

6-1. 전체 평균 하나의 함정

교안 p22가 든 비유가 인상적입니다 — **얼음물과 끓는물에 한 발씩 담근 사람의 평균 체감 온도는 "적당함"** 입니다. 설비 전체 평균 온도가 45도라고 해서 모든 설비가 45도인 것은 아닙니다.

같은 데이터, 전혀 다른 그림

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")

# 전체 평균 하나
print("전체 평균 온도:", round(df["온도"].mean(), 2))

# 냉각기상태로 나눠 보면
print(df.groupby("냉각기상태")["온도"].mean().round(2).to_dict())
```

- ▶ 전체 평균 온도: 45.34
- ▶ {'고장': 54.67, '저하': 45.46, '정상': 35.89}

전체 평균 45.34도는 **어느 상태에도 해당하지 않는 숫자**입니다. 정상은 35.89도, 고장은 54.67도로 **약 19도나 벌어져** 있는데, 전체 평균 하나만 보면 이 차이가 완전히 묻힙니다. 교안의 표현대로 **"의미 있는 그룹으로 나눈 뒤 그룹마다 따로 집계"** 해야 문제가 집중된 곳이 보입니다.

설비 적용

이 데이터는 **냉각기상태가 온도를 좌우한다**는 것을 명확히 보여 줍니다. 정상 35.89 → 저하 45.46 → 고장 54.67로 **상태가 나빠질수록 온도가 계단처럼 올라갑니다**. 거꾸로 읽으면 **온도만 보고도 냉각기 상태를 짐작할 수 있다**는 뜻이고, 이것이 고장 예측 모델의 기본 아이디어입니다.

6-2. 분할 - 적용 - 결합

교안 p24가 groupby의 작동 방식을 3단계로 설명합니다. **판다스의 그룹 집계는 항상 이 구조**입니다.



단계	무엇을	코드의 어느 부분	결과에 미치는 영향
① 분할	그룹으로 나눔 — 고장·저하·정상 세 덩어리	<code>.groupby("냉각기상태")</code>	결과 의 줄 수 를 정합니다
② 적용	각 그룹에서 계산	<code>["온도"].mean()</code>	결과 의 숫자 를 정합니다
③ 결합	그룹별 결과를 한 표로	(자동)	판다스가 알아서 정리합니다

개념

교안 p25의 요약이 실용적입니다 — **"분할이 줄 수, 적용이 숫자를 정합니다."** 그룹이 3개면 3줄, 100개면 100줄이 나옵니다. 그리고 **적용 단계의 함수만 알아 끼우면** `mean·sum·max·min·count` 무엇이든 됩니다. 분할 부분은 그대로 두고 뒤만 바꾸면 되니, 처음 보는 `groupby` 코드도 결과를 미리 그려 볼 수 있습니다.

6-3. 문장처럼 끊어 읽기

메서드 `.groupby("기준열")["집계열"].집계함수()` 그룹으로 나눠 그룹별 통계 내기

정의 왼쪽에서 오른쪽으로 세 부분으로 끊어 읽습니다 — **기준 → 열 → 함수**.

형태 `df.groupby("냉각기상태")["온도"].mean().round(2)`

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

```
result = df.groupby("냉각기상태")["온도"].mean().round(2)
print(result)
```

```
▶ 냉각기상태
▶ 고장 54.67
▶ 저하 45.46
▶ 정상 35.89
▶ Name: 온도, dtype: float64
```

왜 쓰나 교안 p26의 읽는 법이 정확합니다 — **"df를 가지고, 냉각기상태로 그룹을 나눈 뒤, 온도 열을 골라서, 평균을 내라"**. 문장처럼 읽으면 코드가 자연스럽게 해석됩니다.

응용 집계 함수 자리만 바꾸면 다른 통계가 나옵니다. **분할 부분은 그대로**이므로 코드 모양이 거의 같습니다.

```
g = df.groupby("냉각기상태")["온도"]

print("평균 :", g.mean().round(2).to_dict())
print("최댓값:", g.max().to_dict())
print("최솟값:", g.min().to_dict())
print("건수 :", g.count().to_dict())
print("표준편차:", g.std().round(2).to_dict())
```

- ▶ 평균 : {'고장': 54.67, '저하': 45.46, '정상': 35.89}
- ▶ 최댓값: {'고장': 57.8, '저하': 51.2, '정상': 36.4}
- ▶ 최솟값: {'고장': 35.6, '저하': 44.0, '정상': 35.3}
- ▶ 건수 : {'고장': 40, '저하': 40, '정상': 40}
- ▶ 표준편차: {'고장': 3.76, '저하': 1.47, '정상': 0.36}

개념

표준편차를 보면 **고장 3.76, 정상 0.36**으로 10배 넘게 차이 납니다. 16일차에 배운 대로 **정상 설비는 흔들림이 적고 고장 설비는 요동친다**는 것이 숫자로 확인됩니다.

6-4. 집계할 열 고르기 — 한 겹과 두 겹

18일차에 배운 **대괄호 한 겹·두 겹** 규칙이 여기서도 그대로 적용됩니다.

대괄호 겹수에 따른 결과 모양

```
# 한 겹 — 열 하나, 결과는 Series (한 줄짜리 목록)
print(df.groupby("냉각기상태")["온도"].mean().round(2))
print()
# 두 겹 — 열 여러 개, 결과는 DataFrame (표)
print(df.groupby("냉각기상태")[["온도", "진동"]].mean().round(2))
```

```
▶ 냉각기상태
▶ 고장  54.67
▶ 저하  45.46
▶ 정상  35.89
▶ Name: 온도, dtype: float64
▶
▶   온도  진동
▶ 냉각기상태
▶ 고장  54.67  0.69
▶ 저하  45.46  0.61
▶ 정상  35.89  0.55
```

키워드 `numeric_only=True` 숫자 열만 골라 한 번에 집계

정의 열 이름을 일일이 적지 않고 **숫자 열 전체**를 집계합니다. 글자 열 때문에 나는 오류를 막아 줍니다.

형태 `df.groupby("기준열").mean(numeric_only=True)`

```
print(df.groupby("냉각기상태").mean(numeric_only=True).round(2))

▶   온도  진동  압력  냉각효율
▶ 냉각기상태
▶ 고장  54.67  0.69  163.68  20.12
▶ 저하  45.46  0.61  158.49  27.24
▶ 정상  35.89  0.55  160.84  46.96
```

왜 쓰나 열이 수십 개일 때 **전체**를 **한눈에 훑는 용도**로 좋습니다. 어느 열에서 그룹 간 차이가 큰지 빠르게 찾을 수 있습니다.

응용 위 결과를 보면 **냉각효율**이 **20.12 → 27.24 → 46.96**로 **가장 극적으로 갑니다**. 고장일 때 정상의 절반도 안 됩니다. 온도(54.67 vs 35.89)보다도 차이가 뚜렷해서, 냉각기 상태를 판정하는 지표로는 냉각효율이 더 강력하다는 것을 알 수 있습니다.

그룹 간 차이가 큰 열 찾기

```
g = df.groupby("냉각기상태").mean(numeric_only=True)
for col in g.columns:
    spread = g[col].max() - g[col].min()
    print(f"{col:6s} 차이 {spread:7.2f} · 평균 대비 {spread/g[col].mean()*100:5.1f}%")
```

- ▶ 온도 차이 18.78 · 평균 대비 41.4%
- ▶ 진동 차이 0.14 · 평균 대비 22.7%
- ▶ 압력 차이 5.18 · 평균 대비 3.2%
- ▶ 냉각효율 차이 26.83 · 평균 대비 85.3%

주의

열을 생략하고 `df.groupby("A").mean()` 만 쓰면 글자 열 때문에 오류가 나거나 경고가 뜹니다. 숫자 열만 집계하겠다는 의도를 `numeric_only=True` 로 분명히 밝히는 것이 안전합니다.

STEP 7

groupby 실전

여러 키와 size

7-1. 여러 기준으로 그룹 묶기

메서드 `.groupby(["열1", "열2"])` 기준 두 개 이상으로 세분 그룹

정의 그룹 기준을 리스트로 묶으면 **기준들을 조합한 세밀한 그룹**이 만들어집니다.

형태 `df.groupby(["냉각기상태", "운전부하"])["온도"].mean()`

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")

r = df.groupby(["냉각기상태", "운전부하"])["온도"].mean().round(2)
print(r)
```

```
▶ 냉각기상태 운전부하
▶ 고장 고부하 55.51
▶ 저부하 54.05
▶ 저하 고부하 44.07
▶ 저부하 45.58
▶ 정상 고부하 35.89
▶ Name: 온도, dtype: float64
```

왜 쓰나 단일 기준에서는 안 보이던 패턴이 드러납니다. **고장 전체가 높는데 그중에서도 고부하가 더 높다**는 세밀한 관찰이 이상 징후의 단서가 됩니다.

응용 강사님이 든 비유가 기억하기 좋습니다 — **MBTI**입니다. 내향/외향, N/S, T/F, J/P를 조합하면 16가지가 나오듯, 기준을 겹칠수록 그룹이 잘게 나뉩니다. 다만 **기준의 순서가 관점을 바꿉니다**.

```
# 순서를 바꾸면 보이는 것이 달라집니다
a = df.groupby(["냉각기상태", "운전부하"])["온도"].mean().round(2)
b = df.groupby(["운전부하", "냉각기상태"])["온도"].mean().round(2)

print("냉각기 먼저 — 상태별로 묶여 보입니다")
print(a.head(4).to_string())
print()
print("부하 먼저 — 부하별로 묶여 보입니다")
print(b.head(4).to_string())
```

```
▶ 냉각기 먼저 — 상태별로 묶여 보입니다
▶ 냉각기상태 운전부하
▶ 고장 고부하 55.51
▶ 저부하 54.05
▶ 저하 고부하 44.07
▶
▶ 부하 먼저 — 부하별로 묶여 보입니다
▶ 운전부하 냉각기상태
▶ 고부하 고장 55.51
▶ 저하 44.07
▶ 정상 35.89
▶ 저부하 고장 54.05
```

주의

값이 같은 첫 기준은 **한 번만 표시**되고 아래는 빈칸으로 보입니다. 데이터가 없는 것이 아니라 **읽기 좋게 생략**된 것입니다. 위 결과에서 "정상·저부하" 조합이 안 보이는데, 이것은 실제로 해당 조합의 데이터가 없기 때문입니다.

7-2. size와 count — 결측 처리가 다릅니다

구분		
세는 대상	전체 행	비결측 값
결측값	포함	제외
열 선택	불필요	필요 — ["열"]를 앞에 붙임
쓰임	그룹 크기 파악	그룹별 유효 측정 건수

결측이 없으면 둘이 같습니다

```
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

```
print("size :", df.groupby("밸브상태").size().to_dict())
```

```
print("count:", df.groupby("밸브상태")["온도"].count().to_dict())
```

```
print("결측 :", df.isna().sum().sum(), "건 — 그래서 둘이 같습니다")
```

```
▶ size : {'경미': 20, '심각': 19, '정상': 61, '자연': 20}
```

```
▶ count: {'경미': 20, '심각': 19, '정상': 61, '자연': 20}
```

```
▶ 결측 : 0 건 — 그래서 둘이 같습니다
```

개념

교안 p35가 이 차이의 쓸모를 짚습니다 — "size와 count가 크게 차이 나면 그 그룹에 빠진 값이 많다" 는 뜻이고, 측정 누락이 잦은 라인이나 설비를 찾는 단서가 됩니다. 오늘 데이터는 결측이 없어 차이가 안 보이지만, 19일차에 쓴 13_diecasting_shot.csv처럼 결측이 있는 데이터에서는 걸립니다.

주의

교안 p35의 마지막 경고가 중요합니다 — "빠진 데이터를 모르고 평균 내면 결과가 왜곡됩니다." 100번 중 50번만 기록된 라인의 평균을 다른 라인과 같은 무게로 비교할 수는 없습니다. 그룹별 평균을 낼 때는 반드시 건수도 함께 봐야 합니다.

7-3. 그룹 결과에 정렬 이어 붙이기

groupby 결과도 Series나 DataFrame이므로 19일차에 배운 `sort_values`를 그대로 이어 붙일 수 있습니다. 메서드 체이닝이 여기서도 통합니다.

집계 → 정렬 체이닝

밸브상태별 진동 평균을 큰 순으로

```
r = (df.groupby("밸브상태")["진동"]
      .mean()
      .round(3)
      .sort_values(ascending=False))
print(r)
```

▶ 밸브상태

▶ 심각 0.629

▶ 자연 0.621

▶ 경미 0.617

▶ 정상 0.609

▶ Name: 진동, dtype: float64

주의

교안 p33이 해석의 함정을 짚습니다 — "정렬은 어디를 볼지 알려줄 뿐, 왜 그런지는 현장 지식으로" 판단해야 합니다. 위 결과에서 심각(0.629)과 정상(0.609)의 차이는 **0.02밖에 안 됩니다**. 순서상 심각이 맨 위지만, 이 정도 차이가 의미 있는지는 별도 판단이 필요합니다. STEP 6의 냉각기상태별 온도 차이(19도)와 비교하면 확연히 작습니다.

7-4. 오늘의 마지막 과제 — 학생 데이터로 groupby 연습

수업 마지막에 강사님이 **설비 데이터 대신 학생 성적 데이터**로 groupby 문제를 냈습니다. 익숙한 소재로 개념을 잡게 하려는 의도였습니다.

students_groupby_practice.csv

```
import pandas as pd
```

```
df = pd.read_csv("data/students_groupby_practice.csv", encoding="utf-8")
print(df.shape)
print(df.columns.tolist())
print(df.head(3))
```

▶ (60, 6)

▶ ['학년', '반', '이름', '국어', '영어', '수학']

▶ 학년 반 이름 국어 영어 수학

▶ 0 1 A 박채원 76 45 53

▶ 1 1 B 정수아 89 53 55

▶ 2 1 C 최서진 68 51 44

문제	요구	기준 열	핵심
① 전체 학생 수	전체 행 수	없음	<code>len(df)</code>
② 학년별 학생 수	학년으로 묶어 세기	"학년"	<code>.size()</code>
③ 학년·반별 학생 수	두 기준 조합	["학년", "반"]	여러 키
④ 반별 국어 평균	두 기준 + 평균	["학년", "반"]	<code>.mean().round(2)</code>
⑤ 학년별 영어 평균	한 기준 + 평균	"학년" ← 반은 제외	④와 기준이 다릅니다

자주 하는 실수

문제 ④와 ⑤는 기준이 다릅니다. ④는 "각 반의 국어 평균"이라 ["학년", "반"]으로 묶어 12줄이 나오고, ⑤는 "각 학년의 영어 평균"이라 "학년"만으로 묶어 3줄이 나와야 합니다. 문제를 읽을 때 "각 ○○의"에서 ○○가 그대로 groupby의 기준이 됩니다. 제출 코드에서 이 부분이 어긋났는데 부록 B-2에서 다룹니다.

정답 — ④와 ⑤의 줄 수 차이에 주목

② 학년별 학생 수

```
print(df.groupby("학년").size().to_dict())
```

④ 반별 국어 평균 — 두 기준

```
print(df.groupby(["학년", "반"])["국어"].mean().round(2).head(4).to_string())
```

⑤ 학년별 영어 평균 — 한 기준

```
print(df.groupby("학년")["영어"].mean().round(2).to_string())
```

```
▶ {1: 20, 2: 20, 3: 20}
```

```
▶ 학년 반
```

```
▶ 1    A    76.8
```

```
▶     B    78.8
```

```
▶     C    66.0
```

```
▶     D    59.4
```

```
▶ 학년
```

```
▶ 1    64.80
```

```
▶ 2    73.35
```

```
▶ 3    69.90
```

치트시트

오늘 나온 전부 — 한 장 요약

설비 적용

학년별 영어 평균이 **1학년 64.8 → 2학년 73.35 → 3학년 69.9** 로 2학년이 가장 높습니다. 이런 결과를 보면 자연스럽게 "**왜 2학년만 높지?**" 라는 다음 질문이 나옵니다. 설비 데이터도 똑같습니다 — 그룹 집계는 답을 주는 것이 아니라 **다음 질문을 만들어 주는** 도구입니다.

A-1. 빈도 집계

표현	뜻	결과	비고
<code>df["열"].value_counts()</code>	값별 개수	Series	많은 순 자동 정렬
<code>...value_counts(normalize=True)</code>	값별 비율	합이 1	백분율은 $\times 100$
<code>...value_counts(sort=False)</code>	등장 순서로	정렬 안 함	등급 A·B·C 비교에
<code>...value_counts(dropna=False)</code>	결측도 한 종류로	NaN 행 포함	기본은 무시
<code>.round(3)</code>	소수 정리	—	비율은 보통 3자리

A-2. 구간으로 묶기

표현	뜻	주의
<code>pd.cut(열, bins, labels)</code>	수치형 → 구간 범주	경계가 이름표보다 하나 많아야 합니다
<code>bins=[0, 40, 50, 200]</code>	경계 4개 → 구간 3개	양 끝은 실제 범위보다 넉넉하게
<code>labels=["낮음", "보통", "높음"]</code>	구간 이름표	개수는 경계 수 - 1
<code>right=False</code>	왼쪽 닫힘·오른쪽 열림	기본은 반대
묶은 뒤 <code>.value_counts()</code>	구간별 빈도	순서가 핵심 — 묶고 나서 셉니다

A-3. 그룹 집계

표현	뜻	결과 모양
<code>df.groupby("A")["B"].mean()</code>	A로 나눠 B의 평균	Series — 한 줄 목록
<code>df.groupby("A") [["B", "C"]].mean()</code>	두 열을 한 번에	DataFrame — 표
<code>df.groupby("A").mean(numeric_ only=True)</code>	숫자 열 전체	DataFrame
<code>df.groupby(["A", "B"]) ["C"].mean()</code>	두 기준 조합	계층 인덱스
<code>df.groupby("A").size()</code>	그룹별 전체 행 수	결측 포함 · 열 불필요
<code>df.groupby("A")["B"].count()</code>	그룹별 비결측 수	열 지정 필요
집계 후 <code>.sort_values(ascending=False)</code>	큰 값부터 줄 세우기	우선순위 확인

A-4. 집계 함수 바꿔 끼우기

분할 부분은 그대로 두고 뒤의 함수만 바꾸면 다른 통계가 나옵니다. 이것이 `groupby`를 외우기 쉬운 이유입니다.

함수 자리만 갈아 끼우기

```
g = df.groupby("냉각기상태")["온도"]
```

```
g.mean()      # 평균
g.sum()       # 합계
g.max()       # 최댓값
g.min()       # 최솟값
g.count()     # 건수 (비결측)
g.std()       # 표준편차
g.median()    # 중앙값
```

여러 개를 한 번에 — `agg`

```
print(df.groupby("냉각기상태")["온도"].agg(["mean", "max", "std"]).round(2))
```

```
▶              mean    max    std
▶ 냉각기상태
▶ 고장    54.67    57.8    3.76
▶ 저하    45.46    51.2    1.47
▶ 정상    35.89    36.4    0.36
```

A-5. 그대로 쓰는 그룹 리포트 템플릿

복붙해서 쓰는 그룹 리포트

```
import pandas as pd
```

```
def group_report(df, key, value, top=None):  
    # 그룹별 평균·건수를 한 표로 만들고 평균 순으로 정렬  
    r = df.groupby(key).agg(  
        평균=(value, "mean"),  
        건수=(value, "count"),  
    ).round(2).sort_values("평균", ascending=False)  
    print(f"[{key} 기준 {value}]")  
    print(r.head(top) if top else r)  
    return r
```

```
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

```
group_report(df, "냉각기상태", "온도")
```

▶ [냉각기상태 기준 온도]

▶ 평균 건수

▶ 냉각기상태

▶ 고장 54.67 40

▶ 저하 45.46 40

▶ 정상 35.89 40

오류·함정 사전

✗ 이렇게 하면 → 무슨 일이 → ✓ 바로잡기

B-1. 오늘 만나기 쉬운 오류 여덟 가지

✗ 이렇게 하면	무슨 일이 일어나	✓ 바로잡기
<code>df.value_counts()</code> (열 없이)	행 전체 조합을 셈 — 원하는 결과가 아님	<code>df["열"].value_counts()</code>
<code>df["온도"].value_counts()</code>	고유값 62줄 — 의미 없는 표	<code>pd.cut</code> 으로 먼저 묶기
<code>bins=[0,40,50] + labels=3</code> 개	ValueError — 경계와 이름표 개수 불일치	경계는 이름표보다 하나 많게
경계를 최솟값에 딱 맞춤	최솟값이 NaN 이 됨 (왼쪽 열림)	최솟값보다 작은 값을 첫 경계로
<code>.round(1)</code> 로 6% 비율 확인	0.1로 반올림돼 불균형이 가려짐	<code>.round(3)</code> — 비율은 3자리
<code>df.groupby("A").mean()</code> (글자 열 포함)	오류 또는 경고	<code>numeric_only=True</code> 또는 열 지정
<code>groupby(["학년", "반"])</code> (학년별을 물었는데)	12줄 — 반별 결과가 나옴	<code>groupby("학년")</code> — 3줄
건수 없이 그룹 평균만 비교	데이터가 적은 그룹이 과대평가됨	<code>.size()</code> 를 함께 확인

B-2. 제출 코드 교정 — 실제로 실행해 확인한 것

① 문제 ⑤ — 기준 열이 잘못됐습니다

14_01_example_advanced.py의 문제 5는 "각 학년의 영어 점수 평균"을 묻는데, 코드는 ["학년", "반"]으로 묶었습니다. 문제 4와 기준이 같아 반별 결과 12줄이 나옵니다.

Practice/14_01_example_advanced.py 문제 5

```
# x 제출 코드 — 문제 4와 기준이 같습니다
for (grade, classroom), group in df.groupby(["학년", "반"]):
    eng_mean = round(group["영어"].mean(), 2)
    print(f"{grade}학년 {classroom}반 영어 평균: {eng_mean}점")

# → 12줄 출력

# ✓ 학년만 기준으로
for grade, group in df.groupby("학년"):
    print(f"{grade}학년 영어 평균: {round(group['영어'].mean(), 2)}점")

# ✓ 더 간결하게 — 반복문 없이 한 줄
print(df.groupby("학년")["영어"].mean().round(2).to_string())
```

```
▶ 1학년 영어 평균: 64.8점
▶ 2학년 영어 평균: 73.35점
▶ 3학년 영어 평균: 69.9점
▶
▶ 학년
▶ 1      64.80
▶ 2      73.35
▶ 3      69.90
```

개념

문제를 읽을 때 "각 ○○의" 에서 ○○가 그대로 groupby의 기준이 됩니다. "각 반의 국어 평균"이면 ["학년", "반"] (반은 학년마다 있으므로 둘 다 필요), "각 **학년**의 영어 평균"이면 "학년"만입니다. 결과의 줄 수를 미리 세어 보면 확인됩니다 — 학년 3개면 3줄이어야 합니다.

② 반복문 대신 한 줄로

제출 코드는 for 반복문으로 그룹을 돌았습니다. **틀린 것은 아니지만** groupby의 장점을 절반만 쓴 것입니다. 판다스는 반복문 없이 한 줄로 같은 일을 합니다.

제출 코드 (반복문)	한 줄로	줄 수
for ... in df.groupby("학년"): len(group)	df.groupby("학년").size()	3줄 → 1줄
for ...: round(group["국어"].mean(), 2)	df.groupby(["학년", "반"]) ["국어"].mean().round(2)	4줄 → 1줄
for ...:	df.groupby("학년")	3줄 → 1줄

제출 코드 (반복문)	한 줄로	출 수
<code>round(group["영어"].mean(), 2)</code>	<code>["영어"].mean().round(2)</code>	

주의

반복문이 필요한 경우도 있습니다 — 그룹마다 **복잡한 처리**를 하거나 **그룹별로 파일을 따로 저장**할 때입니다. 하지만 **단순 집계는 한 줄이 원칙**입니다. 코드가 짧아질 뿐 아니라 판다스 내부에서 최적화돼 훨씬 빠릅니다. 데이터가 수십만 행이 되면 차이가 눈에 띕니다.

③ 실습 1 — 교안이 지정한 열과 다릅니다

Practice/14_01_example.py 실습 1

```
# x 제출 코드 — 모든 열을 반복문으로
for col in list(df_1.columns):
    print(df_1[col].value_counts())

# → 온도·진동 같은 수치형까지 세어 62줄짜리 표가 나옵니다

# ✓ 교안 요구 — 범주형 열만 골라서
print(df["밸브상태"].value_counts())    # 설비 열
print(df["운전부하"].value_counts())    # 교대 열
```

```
▶ 밸브상태
▶ 정상 61
▶ 지연 20
▶ 경미 20
▶ 심각 19
▶
▶ 운전부하
▶ 고부하 60
▶ 저부하 60
```

개념

모든 열을 도는 접근이 **탐색 단계에서는 유용**합니다. 어떤 열이 범주형인지 한 번에 보이기 때문입니다. 다만 **수치형 열까지 세면 62줄짜리 무의미한 표**가 섞여 나오므로, `select_dtypes(include="object")`로 글자 열만 골라 도는 편이 낫습니다.

권장 — 범주형만 훑기

```
# 범주형 열만 골라 한 번에 훑기
for col in df.select_dtypes(include="object").columns:
    vc = df[col].value_counts()
    print(f"{col:8s} 종류 {len(vc)}가지 · 최다 {vc.index[0]} {vc.iloc[0]}건")
```

- ▶ 냉각기상태 종류 3가지 · 최다 고장 40건
- ▶ 운전부하 종류 2가지 · 최다 고부하 60건
- ▶ 밸브상태 종류 4가지 · 최다 정상 61건
- ▶ result 종류 2가지 · 최다 정상 67건

④ 잘 한 부분

부분	무엇이 좋은가
실습 3의 범위 확인	<code>max()</code> 와 <code>min()</code> 을 먼저 찍고 경계를 정한 것 — 교안이 요구한 순서 그대로입니다
실습 2의 3단계 출력	개수 → 비율 → round를 차례로 보여 준 것 — 변화 과정이 드러납니다
advanced의 다중 키 언패킹	<code>for (grade, classroom), group in ...</code> — 튜플 언패킹을 정확히 썼습니다
구분선 출력	<code>"=" * 150</code> 로 실습마다 구분 — 긴 출력에서 읽기 좋습니다

B-3. 교안 예상 결과가 실데이터와 맞지 않습니다

19일차에 이어 이번에도 교안의 「예상 결과」가 실제와 다릅니다. 강사 배포 코드에도 x 표시가 붙어 있습니다.

실습	교안 예상 결과	실제 값	비고
실습 1	심각 42건이 최다	정상 61건이 최다 (심각은 19건으로 최소)	강사 코드에 x 표시
실습 2	불합격 약 6%	불합격 6% (12/200)	일치
실습 3	보통 43건이 최다	보통 55건이 최다	최다 구간은 맞음

주의

실습 1의 "심각 42건"은 **정반대**입니다 — 실제 심각은 19건으로 가장 적습니다. 실습 3은 최다 구간(보통)은 맞지만 건수가 다릅니다. 교안이 다른 데이터셋을 기준으로 쓰인 것으로 보입니다. **실행 결과가 정답**이니 슬라이드 숫자와 다르다고 자기 코드를 의심하지 마십시오.

실습 하이라이트

제출한 것 · 남은 실습 · 다음 시간

C-1. 오늘 나온 실습 전체

실습	교안 요구	제출	한 줄 평
13_02 실습 5·6·7	정렬·필터연결·워크플로우	강사 시연	19일차 미완 분량을 오늘 마무리
14_01 실습 1	value_counts로 빈도 세기	제출	모든 열을 돌아 수치형까지 셈
14_01 실습 2	비율과 불균형 데이터	제출	개수→비율→round 3단계 — 좋음
14_01 실습 3	구간으로 묶어 세기	제출	범위 확인 후 경계 설정 — 순서 정확
groupby 종합	학생 데이터 5문제	제출	문제 5의 기준 열이 어긋남
14_01 실습 4~7	유압 데이터 groupby	미출제	다음 시간 — 아래 미리보기

C-2. 다음 시간 실습 미리보기 — 실습 4~7

교안 14_01의 실습 4~7은 유압 데이터로 **groupby**를 본격 적용하는 문제입니다. 오늘 배운 것으로 다 풀 수 있으니 미리 해 보면 좋습니다.

실습 4~6 미리보기

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")

# 실습 4 — 기준 → 열 → 함수
print(df.groupby("냉각기상태")["압력"].mean().round(2).to_dict())
print(df.groupby("밸브상태")["온도"].max().to_dict())
print(df.groupby("운전부하").size().to_dict())

# 실습 5 — 집계 후 정렬
print(df.groupby("밸브상태")["진동"].mean().round(3)
      .sort_values(ascending=False).to_dict())

# 실습 6 — 두 기준 조합 + 건수
print(df.groupby(["냉각기상태", "운전부하"])["진동"].mean().round(3).to_dict())
print(df.groupby(["냉각기상태", "운전부하"]).size().to_dict())
```

- ▶ {'고장': 163.68, '저하': 158.49, '정상': 160.84}
- ▶ {'경미': 57.1, '심각': 57.6, '정상': 57.8, '자연': 57.5}
- ▶ {'고부하': 60, '저부하': 60}
- ▶ {'심각': 0.629, '자연': 0.621, '경미': 0.617, '정상': 0.609}

주의

실습 6의 마지막 줄이 중요합니다 — 교안 p38이 "건수가 적은 조합의 평균은 신중히 해석" 하라고 합니다. 오늘 확인했듯 **정상·저부하 조합은 데이터가 아예 없습니다**. 조합별 건수를 함께 보지 않으면 그 사실을 모른 채 평균만 비교하게 됩니다.

C-3. 실습 7 — 빈도와 그룹 집계 종합

교안 실습 7은 오늘 배운 두 축(빈도·그룹)을 한 흐름으로 연결하는 문제입니다. 교안 p40이 요구하는 산출물은 코드가 아니라 **세 문장**입니다.

실습 7 — 빈도 + 필터 + 그룹

```
import pandas as pd
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")

# ① value_counts 로 구성과 비율 파악
print("구성:", df["냉각기상태"].value_counts().to_dict())
print("정상·고장 비율:",
      df["result"].value_counts(normalize=True).round(3).to_dict())

# ② 고장 행만 걸러 상태별 건수
bad = df[df["result"] == "고장"]
print("고장 중 냉각기상태별:", bad["냉각기상태"].value_counts().to_dict())
print("고장 중 밸브상태별 :", bad["밸브상태"].value_counts().to_dict())
```

- ▶ 구성: {'고장': 40, '저하': 40, '정상': 40}
- ▶ 정상·고장 비율: {'정상': 0.558, '고장': 0.442}
- ▶ 고장 중 냉각기상태별: {'고장': 18, '정상': 18, '저하': 17}
- ▶ 고장 중 밸브상태별 : {'심각': 14, '경미': 14, '자연': 13, '정상': 12}

자주 하는 실수

두 번째 출력을 그대로 읽으면 **"고장 중 밸브 심각이 14건으로 가장 많다"** 는 결론이 나옵니다. 그런데 이것은 틀린 해석입니다. 밸브 정상인 행이 원래 61건으로 압도적으로 많으니, 건수만 비교하면 안 됩니다. **건수가 아니라 그룹 안에서의 비율로** 봐야 합니다.

비율로 다시 보면 그림이 뒤집힙니다

```
# 건수가 아니라 "그룹 안에서의 비율"로
print("[냉각기상태별 고장률]")
for state, g in df.groupby("냉각기상태"):
    print(f"    {state}: {(g['result']=='고장').mean()*100:.1f}%    (n={len(g)})")

print("[밸브상태별 고장률]")
for state, g in df.groupby("밸브상태"):
    print(f"    {state}: {(g['result']=='고장').mean()*100:.1f}%    (n={len(g)})")
```

- ▶ [냉각기상태별 고장률]
- ▶ 고장: 45.0% (n=40)
- ▶ 저하: 42.5% (n=40)
- ▶ 정상: 45.0% (n=40)
- ▶ [밸브상태별 고장률]
- ▶ 경미: 70.0% (n=20)
- ▶ 심각: 73.7% (n=19)
- ▶ 정상: 19.7% (n=61)
- ▶ 지연: 65.0% (n=20)

설비 적용

비율로 보니 **완전히 다른 그림**이 나옵니다. 냉각기상태는 고장률이 42.5~45%로 세 상태가 거의 같습니다 — 온도는 상태별로 19도나 갈랐는데 정작 고장 여부와는 관계가 없습니다. 반면 밸브상태는 **정상 19.7% 대 심각 73.7%로 네 배 가까이 벌어집니다**. 이 데이터에서 설비 고장을 좌우하는 것은 냉각기가 아니라 **밸브**입니다. 건수만 보고 넘어갔다면 정반대 결론을 냈을 상황입니다.

C-4. 다음 시간 예고 — 14_02 그룹별 통계량

교안	주제	오늘과의 연결
14_02	그룹별 통계량 — 평균을 넘어 분산·표준편차까지	오늘 <code>.mean()</code> 만 썼던 자리에 <code>.std()</code> · <code>.agg()</code>
14_02	여러 통계를 한 번에	<code>.agg(["mean", "std", "count"])</code>
14_03	상관관계와 통합 리포트	교안 p6의 세 번째 질문 — <code>corr()</code>
14_03	분석 결과를 리포트로	오늘까지 배운 것의 종합

연결

교안 p40의 마지막 문장이 다음 시간을 예고합니다 — "**다음 시간에는 그룹별 비교를 평균을 넘어 분산·표준편차까지 확장**" 합니다. 16일차 넘파이에서 배운 "**평균은 중심, 표준편차는 흩어짐**" 이 그룹 단위로 올라오는 것입니다. 오늘 STEP 6에서 냉각기상태별 표준편차가 3.76 대 0.36으로 갈리는 것을 이미 봤는데, 그 해석을 본격적으로 다룹니다.

오늘 넘은 선 — 거르기에서 요약하기로

지금까지는 원하는 행을 골라내는 도구를 배웠습니다. 오늘부터는 골라낸 것을 숫자 몇 개로 압축하는 도구입니다.

조건을 종류마다 거는 대신 `value_counts` 한 줄, 그룹마다 필터를 거는 대신 `groupby` 한 줄로 끝납니다.

그리고 오늘 가장 값진 한 장면은 **전체 평균 45.34**도였습니다. 어느 상태에도 해당하지 않는 이 숫자가, 그룹으로 나누는 순간 정상 35.89 · 저하 45.46 · 고장 54.67로 갈렸습니다. **평균 하나로는 아무것도 알 수 없다**는 것을 눈으로 확인한 셈입니다.

오늘 할 수 있게 된 것	확인 질문	못 하면
범주형과 수치형을 구분한다	온도에 <code>value_counts</code> 를 바로 쓰면?	STEP 2
값별 개수를 한 줄로 센다	출력은 무슨 순서로 정렬되나?	STEP 3
개수를 비율로 바꾼다	불균형 데이터의 함정은 무엇인가?	STEP 4
수치형을 구간으로 묶는다	경계와 이름표의 개수 관계는?	STEP 5
분할-적용-결합을 설명한다	결과의 줄 수를 정하는 것은 어느 단계인가?	STEP 6
그룹별 통계를 낸다	<code>groupby("A")["B"].mean()</code> 을 문장으로 읽으면?	STEP 6
여러 기준으로 묶는다	기준의 순서를 바꾸면 무엇이 달라지나?	STEP 7
<code>size</code> 와 <code>count</code> 를 구분한다	둘이 다르면 무엇을 뜻하나?	STEP 7

복습 루틴 — 35분



다섯 줄로 오늘 전부

② 세 도구를 한 파일에서 확인

```
df = pd.read_csv("data/14_hydraulic.csv", encoding="utf-8")
```

```
print("[빈도]", df["밸브상태"].value_counts().to_dict())
```

```
print("[비율]", df["result"].value_counts(normalize=True).round(3).to_dict())
```

```
band = pd.cut(df["온도"], bins=[0, 40, 50, 200], labels=["낮음", "보통", "높음"])
```

```
print("[구간]", band.value_counts().to_dict())
```

```
print("[그룹]", df.groupby("냉각기상태")["온도"].mean().round(2).to_dict())
```

```
print("[건수]", df.groupby("냉각기상태").size().to_dict())
```

- ▶ [빈도] {'정상': 61, '지연': 20, '경미': 20, '심각': 19}
- ▶ [비율] {'정상': 0.558, '고장': 0.442}
- ▶ [구간] {'낮음': 41, '보통': 40, '높음': 39}
- ▶ [그룹] {'고장': 54.67, '저하': 45.46, '정상': 35.89}
- ▶ [건수] {'고장': 40, '저하': 40, '정상': 40}

단계	무엇을	왜
① 부록 A 훑기 (5분)	A-1·A-2·A-3 세 표를 눈으로 넘긴다	빈도·구간·그룹을 머리에 배치
② 세 도구 (10분)	위 코드를 실행해 결과를 확인한다	한 번씩 손으로 쳐 봐야 붙는다
③ 실습 4~7 (15분)	C-2·C-3을 보지 않고 짜 본다	다음 시간에 바로 쓰입니다
④ 부록 B 교정 (5분)	문제 5 기준 열 · 반복문을 한 줄로	제출본 완성도를 올린다

설비 적용

취업 관점에서 groupby는 **Pandas 면접의 단골**입니다. "라인별 불량률을 구해 보세요" 같은 문제가 그대로 나옵니다. **기준 → 열 → 함수** 순서와 **분할·적용·결합** 3단계를 말로 설명할 수 있으면 충분합니다.

다음 시간에는 그룹별 비교를 **평균에서 분산·표준편차까지** 확장합니다.